

医院专家门诊预约管理系统——答辩代码说明

本文依据当前仓库源码整理，目标是在答辩时能够说明“系统怎么分层、每个类负责什么、关键函数如何工作、一次操作如何贯穿界面与业务层、代码有哪些边界”。

1. 一分钟项目介绍

本项目是一个基于 **C++17 + Qt Widgets** 的医院专家门诊预约管理系统。系统以 **Hospital** 作为业务数据中心，统一管理医生集合与预约集合；以 **Human**、**Doctor**、**Patient**、**Timeslot**、**Appointment** 表达领域对象；以 **MainWindow** 组织首页、挂号、预约查询、医生管理四个页面；数据通过 **QDataStream** 写入两个二进制文件，并使用 **QSaveFile** 降低保存过程中产生残缺文件的风险。

系统已经实现的主要能力：

- 医生新增、编辑、删除、查询和排班管理；
- 预约创建、查询、改约、退号；
- 医生编号、预约号唯一性检查；
- 排班冲突、时段容量、过期时段、同一患者同一时段重复预约等校验；
- 删除医生时级联删除该医生的预约；
- 医生和预约数据的加载、保存、导入、导出；
- 首页统计、近期预约和科室分布展示；
- 模型层自动测试。

1.1 项目特色与优势（建议口述 60–90 秒）

本项目的第一个特点是**架构清晰、界面与业务分离**。虽然它不是 Web 项目，没有浏览器前端和服务端，但在代码结构上实现了类似的职责分离：`ui/` 负责页面展示和用户交互，`src/` 负责医生、排班、预约等业务对象和规则，界面变化不会直接改变核心业务逻辑。第二个特点是**面向对象建模完整**，通过 `Human` 基类复用医生和患者的公共属性，通过 `Hospital` 统一管理医生和预约数据。第三个特点是**业务校验比较完善**，能够处理编号重复、排班冲突、号源容量、重复预约、过期时段和删除医生后的级联处理。第四个特点是**数据可靠性和可测试性较好**，使用 `QSaveFile` 降低文件写入中断的风险，并且模型层能够脱离图形界面单独测试。最后，系统使用 Qt 信号与槽完成页面跳转和状态刷新，模块之间耦合较低，后续增加新页面或更换存储方式也比较方便。

答辩时可以将优势概括为以下五点：

1. **界面层与业务层分离**：`ui/` 只负责交互，`src/` 集中实现数据模型和核心规则。更准确的说法是“分层设计”或“界面与业务分离”，不要直接称为 Web 式前后端分离。
2. **领域模型清晰**：人员、医生、患者、排班、预约、医院各有独立类，类之间的继承、聚合和关联关系与现实业务对应。
3. **数据一致性较强**：界面层负责及时提示，`Hospital` 再执行最终业务校验；删除医生会级联处理预约，预约号由统一入口维护。
4. **低耦合、易扩展**：各页面共享 `Hospital`，通过信号与槽通信，不直接相互控制；以后可以增加统计页面、数据库存储或登录权限模块。
5. **可靠且可验证**：`QSaveFile` 提供单文件原子替换，关键修改具有回滚处理；模型层测试覆盖排班、容量、防重复和文件读写等规则。

如果老师追问“前后端是怎么分离的”，可以回答：

本项目是单机 Qt 桌面程序，因此没有网络意义上的前端和后端。我所说的分离是代码职责上的分层：页面和弹窗是表现层，`Hospital` 及各领域类是业务与数据层。页面通过公开接口调用业务层，而不是直接操作文件；文件读写也集中在 `Hospital` 中。这样的结构使界面、业务规则和数据保存相对独立。

1.2 八分钟答辩时间建议

环节	建议时间	主要内容
项目背景与目标	40 秒	解决医生、排班和预约的统一管理问题。
功能与项目特色	1 分 20 秒	四个页面、分层设计、业务校验、持久化与测试。
架构和核心类	1 分 30 秒	Hospital、继承关系、排班与预约关联。
关键流程或代码	1 分 30 秒	重点讲一次挂号如何经过页面校验、业务校验和文件保存。
系统演示	2 分钟	排班、挂号、防重复、查询或退号，选择一条完整主线。
不足与总结	1 分钟	改约统一校验、数据库扩展、权限与隐私等。
机动时间	20 秒	用于界面切换或现场停顿。

八分钟内不建议逐个讲所有类和函数。正文后续的详细说明主要用于回答老师追问，正式陈述只需突出 Hospital、Timeslot、Appointment 和一次完整挂号流程。

2. 技术栈与工程组织

项目	实现
语言标准	C++17
GUI	Qt Widgets、Qt Designer <code>.ui</code> 文件
容器	<code>QList</code> 、 <code>QStringList</code> 、 <code>QMap</code>
日期时间	<code>QDate</code> 、 <code>QTime</code> 、 <code>QDateTime</code>
持久化	<code>QFile</code> 、 <code>QSaveFile</code> 、 <code>QDataStream</code>
构建	qmake，工程文件为 <code>09025117.pro</code>
测试	自定义控制台测试入口 + <code>assert</code>

目录职责：

路径	作用
<code>src/</code>	与界面无关的领域模型和业务管理
<code>ui/pages/</code>	主窗口中的四个功能页面
<code>ui/dialogs/</code>	医生、排班、预约详情编辑弹窗
<code>ui/widgets/</code>	自定义日历控件
<code>ui/*.h</code>	全局界面常量、下拉数据、保存辅助函数
<code>resources/</code>	QSS 样式和 Qt 资源清单
<code>test/</code>	模型层测试
<code>tools/</code>	初始化医生数据的辅助程序

`09025117.pro` 使用 `BUILD_TESTS` 条件区分两种构建目标：默认构建 GUI；传入 `BUILD_TESTS=1` 时不编译界面，改为编译 `hospital_tests` 控制台测试程序。这使模型层可以脱离 GUI 单独验证。

3. 总体架构

```
1  main.cpp
2      | 创建 QApplication 和 MainWindow
3      ▼
4  MainWindow ----- 持有唯一 Hospital 实例
5      |
6      ├── DashboardPage          |  QList<Doctor>
7      ├── RegisterPage           |  QList<Appointment>
8      ├── AppointmentsPage
9      └── DoctorsPage
10         |
11         ├── DoctorDialog
12         └── TimeslotDialog
13
14  AppointmentsPage -- AppointmentDialog
15
16  Human
17      ├── Doctor -- 聚合 QList<Timeslot>
18      └── Patient
19
20  Appointment -- 保存 Patient、doctorId、日期和 Timeslot
```

这里使用的是“轻量分层”设计：

1. **模型/业务层**： `src/` 中的类保存数据并执行核心规则。
2. **界面层**：页面与弹窗负责收集输入、显示结果和触发业务操作。
3. **持久化层**：当前未单独拆类，而是集中在 `Hospital` 的 `load/save/import/export` 中。

所有页面都持有同一个 `Hospital&` 引用，因此页面之间共享同一份内存数据，不需要复制或建立全局变量。主窗口在切页时调用目标页面的 `refresh()`，让界面从共享模型重新读取最新数据。

4. 核心领域类

4.1 Human：人员基类

文件：`src/human.h`、`src/human.cpp`

`Human` 抽取医生和患者共有的姓名、年龄、性别，体现面向对象中的继承和代码复用。

成员变量：

- `QString name`：姓名；
- `int age`：年龄；
- `Gender gender`：性别，强类型枚举，仅含 `Male`、`Female`。

函数：

函数	作用与关键点
<code>Human(QString, int, Gender)</code>	初始化公共属性；年龄小于 0 时抛出 <code>std::invalid_argument</code> 。
<code>getName/getAge/getGender</code>	返回对应属性，不修改对象，所以声明为 <code>const</code> 。
<code>setName/setGender</code>	修改姓名或性别。
<code>setAge</code>	修改年龄，并再次执行非负校验，防止构造后写入非法年龄。

答辩要点：选择 `enum class` 而不是普通枚举，可以避免枚举值泄漏到外层命名空间，并获得更严格的类型检查。

4.2 Doctor：医生及其每周排班

文件：`src/doctor.h`、`src/doctor.cpp`

`Doctor : public Human`，在人员公共信息之上增加医生编号、职称、科室和排班。

成员变量：

- `doctorId`：医生唯一标识；

- `title`：职称；
- `department`：科室；
- `QList<Timeslot> schedule`：每周周期性出诊时段。

关键函数：

函数	作用与算法
<code>addTimeslot</code>	先调用 <code>isConflict</code> 检查新时段是否与同一天已有时段重叠；冲突则抛异常，否则加入并排序。
<code>removeTimeslot</code>	线性查找相等时段并删除；找不到时通过 <code>qWarning()</code> 输出诊断信息。
<code>isConflict</code>	遍历排班，只要任一时段的 <code>overlaps()</code> 为真就判定冲突。它是私有函数，因为冲突判断是添加排班的内部实现。
<code>setSchedule</code>	整体替换排班并排序，供排班弹窗保存使用。
<code>getSchedule</code>	返回 <code>const QList<Timeslot>&</code> ，避免复制，同时阻止调用方直接修改容器。
其余 getter/setter	读取或修改职称、科室等基本信息。医生编号没有 setter，界面编辑时也锁定编号。

复杂度：添加时段的冲突检查是 $O(n)$ ，随后排序约为 $O(n \log n)$ ；数据规模较小时足够直观可靠。

4.3 Patient：患者信息

文件：`src/patient.h`、`src/patient.cpp`

`Patient`：public `Human`，新增：

- `patientId`：身份证号；
- `phoneNumber`：联系电话。

构造函数负责初始化，getter/setter 负责读取与修改。身份证和手机号的格式校验不在该类中，而是在挂号页面提交时执行；因此 `Patient` 本身是较轻的数据对象。

4.4 Timeslot：每周出诊时段

文件：`src/timeslot.h`、`src/timeslot.cpp`

成员变量：星期、开始时间、结束时间、容量；默认容量由静态变量 `default_capability = 5` 提供。

函数	作用与边界语义
构造函数	要求 <code>startTime < endTime</code> 且容量大于 0，否则抛异常。
<code>operator<</code>	先按星期、再按开始时间排序，用于保证医生排班顺序稳定。
<code>operator==</code>	比较星期、开始和结束时间，不比较容量；业务上“同一个时段”由时间范围决定。
<code>contains</code>	判断一个日期时间是否落在该星期的 <code>[开始时间, 结束时间)</code> 内。结束点不包含，可避免相邻时段在边界处重复命中。
<code>overlaps</code>	同一天且满足 <code>start < other.end && other.start < end</code> 才重叠； <code>09:00-10:00</code> 与 <code>10:00-11:00</code> 不冲突。
getter	返回星期、起止时间和容量。

`overlaps` 的条件是区间相交的标准判断，使用严格小于号使首尾相接的排班合法。

4.5 Appointment：一次具体预约

文件：`src/appointment.h`、`src/appointment.cpp`

成员变量：预约号、医生编号、完整患者对象、症状、具体日期和时段。

设计特点：

- 保存 `doctorId` 而不是完整医生对象，避免医生资料在预约中形成副本并与主数据不一致；
- 保存完整 `Patient`，使每条预约保留当次挂号填写的患者资料；
- 构造时检查具体日期的星期是否等于 `Timeslot.dayOfWeek`，否则抛出异常；
- `Hospital` 被声明为友元，只有它能调用私有的 `setAppointId`，集中维护预约号唯一性。

函数	作用
<code>hasEnded(reference)</code>	以“预约日期 + 时段结束时间”组成时间点，小于等于参考时间即完成。
<code>hasStarted(reference)</code>	当前实现为开始时间严格早于参考时间；恰好等于开始时刻时仍返回 <code>false</code> 。
<code>setDate/setTimeslot/setSymptom</code>	供改约功能更新可变信息。
<code>setAppointId</code>	私有，只允许 <code>Hospital::changeAppointmentId</code> 间接修改。
getter	返回预约各字段。当前 <code>Patient</code> 和 <code>Timeslot</code> 按值返回。

4.6 `Hospital`：业务管理核心

文件：`src/hospital.h`、`src/hospital.cpp`

`Hospital` 同时承担三项职责：保存医生/预约集合、执行跨对象业务规则、完成文件持久化。它是系统中最重要的一类。

医生管理

函数	实现
<code>addDoctor</code>	用 <code>findDoctor</code> 检查编号，重复则抛异常，否则加入列表。
<code>findDoctor</code>	线性查找并返回列表中对象的指针，未找到返回 <code>nullptr</code> 。
<code>removeDoctor</code>	删除目标医生，然后从后向前遍历预约集合，级联删除该医生的所有预约。反向删除可避免索引前移导致漏删。

预约管理与规则顺序

`addAppointment` 按以下顺序防守：

- 1. 医生必须存在；
- 2. 妇产科仅接受女性患者；
- 3. 默认不允许预约已经开始的时段；
- 4. 预约号不能重复；
- 5. 同一身份证号不能在同一日期、同一时段重复预约；
- 6. 所选时段必须属于该医生的排班；
- 7. 该医生在具体日期和时段的预约数不能达到容量上限；
- 8. 全部通过后才追加到 `appointments`。

这种设计把关键规则放在业务层，即使将来替换界面，仍不能绕过这些约束。参数 `allowPast` 主要为特殊数据场景预留；普通调用使用默认值 `false`。

其他预约函数：

函数	实现
<code>findAppointment</code>	按预约号线性查找，返回指针或 <code>nullptr</code> 。
<code>removeAppointment</code>	找到索引后删除并返回 <code>true</code> ，否则返回 <code>false</code> 。
<code>changeAppointmentId</code>	校验原预约存在、新编号非空且不重复，再通过友元权限修改私有预约号。
<code>countAppointments</code>	统计指定医生、日期和时段的预约数，用于计算余号。
<code>hasPatientAppointment</code>	检查同一身份证号在指定日期、时段是否已有预约；当前规则跨医生生效。
<code>nextAppointmentId</code>	生成 <code>A + yyMMdd + 三位序号</code> ，如 <code>A260907001</code> ；从 001 到 999 找当天第一个未使用编号。

文件持久化

默认数据目录是 `QCoreApplication::applicationDirPath() + "/data"`。测试可向构造函数传入临时路径，因此不会污染真实数据。

函数	作用
<code>load</code>	文件存在则读取；文件不存在则清空相应集合并视为正常。先读医生，再读预约，因为预约通过医生编号建立关联。
<code>save</code>	依次调用 <code>saveDoctors()</code> 与 <code>saveAppointments()</code> 。
<code>loadDoctors</code>	先读记录数，再逐项反序列化医生和其排班，全部成功后整体赋给成员列表，避免半份医生数据进入内存。
<code>loadAppointments</code>	反序列化预约；引用不存在医生的记录、妇产科男性患者记录会被跳过。
<code>saveDoctors/saveAppointments</code>	用 <code>QDataStream</code> 顺序写字段，以 <code>QSaveFile::commit()</code> 原子替换单个目标文件。
<code>importData</code>	在临时 <code>Hospital</code> 中读取外部两份文件；成功后替换当前内存数据并保存到当前工作路径，保存失败则恢复旧内存数据。
<code>exportData</code>	在目标文件夹创建一个临时逻辑上的 <code>Hospital</code> ，复制当前集合后保存为标准文件名。

二进制格式是“字段顺序协议”，读取顺序必须与写入顺序完全一致：

```

1 doctors.dat
2 医生数量
3   └─ 姓名、年龄、性别、编号、职称、科室、排班数量
4     └─ 星期、开始时间、结束时间、容量
5
6 appointments.dat
7 预约数量
8   └─ 预约号、医生编号、患者姓名/年龄/性别/身份证/电话、症状、日期、星期、起止时间、容量

```

5. 界面类和函数

5.1 `main.cpp` 与 `MainWindow`

`main.cpp` 创建 `QApplication`，创建并显示 `MainWindow`，最后进入 Qt 事件循环。

`MainWindow` 是组合根：它持有唯一的 `Hospital hospital_`，启动时先 `load()`，再构造各页面，从而避免页面初次显示空数据。

函数	作用
构造函数	初始化 UI、侧栏导航、加载数据、创建页面、连接信号槽、加载资源中的 QSS。
<code>buildPages</code>	按首页、挂号、查询、医生管理的顺序加入 <code>QStackedWidget</code> ，并连接跨页面跳转信号。
<code>onNavChanged</code>	切换栈页面，通过 <code>QMetaObject::invokeMethod</code> 调用当前页的 <code>refresh()</code> 。
<code>refreshPages</code>	导入数据后刷新全部页面。
<code>importData</code>	选择两份 <code>.dat</code> 文件，二次确认后调用业务层导入。
<code>exportData</code>	选择目录，发现同名文件时询问是否覆盖，再调用业务层导出。
<code>hospital()</code>	返回共享业务对象引用。

`QMetaObject::invokeMethod` 的优点是主窗口不必为每一种页面写强制类型转换；只要页面暴露名为 `refresh` 的槽即可统一刷新。

5.2 DashboardPage：首页概览

函数	作用
refresh	依次刷新统计、近期预约、科室分布。
refreshStatistics	计算医生总数、预约总数、今日预约数和今日剩余号源。
refreshUpcomingAppointments	过滤今天及以后预约，按日期和开始时间排序，最多显示 6 条。
refreshDepartments	用 QMap 聚合各科室医生数，再按人数降序、科室名升序显示。

三个快捷按钮发出 navigateRequested(pageIndex)；主窗口收到后切换侧栏。首页不直接依赖其他页面，降低页面之间的耦合。

5.3 DoctorsPage：医生管理

函数	作用
<code>refresh</code>	重建科室过滤器并填充医生表格。
<code>fillDeptFilter</code>	合并预置科室与数据中自定义科室，保留当前选择。
<code>populateTable</code>	按关键词和科室筛选医生，显示基本信息、排班数量和操作按钮。
<code>nextFreeDoctorId</code>	从 <code>D001</code> 起寻找未占用编号。
<code>scheduleText</code>	将排班格式化为 tooltip 文本。
<code>addRowActions</code>	为每行动态创建排班、编辑、删除、预约四个按钮并连接 lambda。
<code>onAdd</code>	打开 <code>DoctorDialog</code> ，构造医生，调用 <code>Hospital::addDoctor</code> ，成功后保存和刷新。
<code>editDoctor</code>	对话框预填数据；编号不可改，只更新其他字段并保存。
<code>deleteDoctor</code>	二次确认后删除；业务层同时级联删除预约。
<code>scheduleDoctor</code>	打开 <code>TimeslotDialog</code> 编辑排班并保存。
<code>registerWithDoctor</code>	发出携带医生编号的跨页面信号，主窗口跳转到挂号页并预选医生。
<code>adjustColumnWidths</code>	内容自适应后，将剩余宽度按比例分给数据列，操作列保持可用。

5.4 RegisterPage：预约挂号

函数	作用
fillDoctors	重建可搜索医生下拉框，尽量保留或指定医生。
currentDoctorId	只有输入内容与某个下拉项完全一致时才返回编号，防止把自由输入误当成有效医生。
updateDoctorCard	显示医生职称、科室、性别、年龄和编号。
refreshSlots	按所选日期过滤医生排班，调用 countAppointments 计算余号；过期或满额时段被禁用。
updateSubmitState	仅在医生有效且选中了可用时段时启用提交按钮。
submitAppointment	完成输入格式校验、生成预约号、二次确认、业务层校验、保存；保存失败则从内存删除刚新增预约作为回滚。
clearPatientForm	成功后清空患者输入，但保留医生和日期以便连续操作。

输入校验包括：姓名非空、身份证满足 `^[0-9]{17}[0-9X]$`、手机号满足 `^1\d{10}$`、症状非空。这里验证的是格式，不是身份证校验码或真实号码归属。

5.5 AppointmentsPage：预约查询与管理

函数	作用
<code>fillDoctors</code>	填充“全部医生 + 各医生”过滤项并保留选择。
<code>populateTable</code>	按医生、关键词、是否显示已完成预约筛选；再按日期、开始时间、医生名、预约号排序。
<code>addRowActions</code>	所有记录有“详情”；未完成记录额外显示“退号”。
<code>showDetails</code>	打开预约详情；允许修改时更新日期、时段、症状，跨日期时重新生成预约号，保存失败则恢复旧值。
<code>cancelAppointment</code>	已结束预约不能退号；确认后删除并保存，失败则重新加载磁盘数据恢复内存。
<code>resetFilters</code>	恢复全部医生、隐藏已完成、清空关键词。
<code>adjustColumnWidths</code>	按内容和可用空间调整列宽。

身份证与手机号列在表格中被隐藏，但数据仍保存在表格模型中并可在详情里查看。这是界面简洁性的处理，不等于数据脱敏或权限控制。

5.6 三个弹窗和自定义控件

DoctorDialog

- 新增模式可预填建议编号；编辑模式通过 `setDoctor` 回填数据并锁定编号。
- `doctor()` 根据当前表单构造一个新的 `Doctor` 值对象。
- `onOkClicked()` 校验姓名、编号、职称和科室非空。

TimeslotDialog

- 构造时复制医生排班到 `slots_`，因此点击取消不会影响原医生；
- `addSlot()` 校验起止时间并用 `overlaps` 检查冲突；
- `removeSelected()` 只修改临时列表；
- `saveSchedule()` 才调用 `doctor_.setSchedule(slots_)` 整体提交。

这是一种简单的“先编辑副本，确认后提交”策略。

AppointmentDialog

- 初始为只读详情模式；已完成预约的“修改”按钮被禁用；
- `enterEditMode()` 动态将按钮组变为保存/取消，并开放日期、时段、症状；
- `refreshSlots()` 只展示医生当天、尚未开始、有剩余容量的排班。计算改约目标余号时，会减去当前预约本身，避免原时段被自己占用而误判满额；
- `acceptChanges()` 做非空和确认检查，真正修改由 `AppointmentsPage::showDetails()` 完成。

AppointmentCalendar

重写 `QCalendarWidget::paintCell`：非当前月份日期不绘制普通内容，超出可选范围的日期用灰色显示，提升日期范围的视觉可读性。

6. 四条关键调用链

6.1 程序启动

```
1  main
2  → QApplication
3  → MainWindow 构造
4  → hospital_.load()
5  → loadDoctors()
6  → loadAppointments()
7  → buildPages()
8  → 各页面 refresh()
9  → 显示窗口并进入事件循环
```

必须先加载医生再加载预约，因为预约只保存医生编号，反序列化时需要先找到对应医生。

6.2 新增预约

```
1  用户选择医生/日期/时段并填写患者资料
2  → RegisterPage::submitAppointment
3  → 页面做格式校验
4  → Hospital::nextAppointmentId
5  → 构造 Patient、Appointment
6  → Hospital::addAppointment 执行业务规则
7  → persistHospital
8  → Hospital::save
9  → QSaveFile 写 doctors.dat / appointments.dat
10 → 清空表单并刷新余号
```

界面层校验提升用户体验，业务层校验保证数据规则；两层校验并不重复浪费，而是分别解决“及时提示”和“不可绕过”两个问题。

6.3 修改预约

```
1  AppointmentsPage 双击或点击详情
2  → AppointmentDialog 显示只读信息
3  → 进入编辑模式
4  → refreshSlots 计算可选时段
5  → 对话框返回新日期/时段/症状
6  → 页面暂存旧值
7  → 修改 Appointment
8  → 跨日期则 Hospital::changeAppointmentId
9  → 保存成功：刷新表格
10 → 保存失败：恢复原编号、日期、时段、症状
```

6.4 删除医生

```
1  DoctorsPage::deleteDoctor
2  → 用户二次确认
3  → Hospital::removeDoctor
4  → 删除医生
5  → 反向遍历并删除其全部预约
6  → persistHospital
7  → 刷新医生表格
```

7. 信号与槽如何解耦界面

项目大量使用 Qt 信号槽：按钮点击、下拉框变化、日期选择、表格双击都通过 `connect` 连接到槽函数或 `lambda`。

跨页面通信的典型例子：

- `DashboardPage` 只发出 `navigateRequested(index)`，不知道主窗口如何切页；
- `DoctorsPage` 发出 `requestRegisterAppointment(doctorId)`；
- `MainWindow` 接收信号，切到挂号页并调用 `RegisterPage::selectDoctor`。

这样页面不直接持有其他页面指针，导航责任集中在主窗口，耦合较低。

8. 异常、返回值与回滚策略

代码根据错误性质采用三种方式：

1. **违对象不变量或业务规则**：抛 `std::invalid_argument`，如负年龄、时段非法、编号重复、容量已满；界面捕获并弹窗。
2. **查找/删除类正常失败**：返回 `nullptr` 或 `false`，如医生不存在、预约不存在。
3. **I/O 失败**：`load/save/import/export` 返回 `bool`，界面显示统一错误提示。

局部回滚例子：

- 新增预约后保存失败，删除刚加入的预约；
- 修改预约保存失败，恢复旧编号、日期、时段、症状；
- 退号保存失败，重新从磁盘加载；
- 导入后保存失败，恢复导入前的内存集合。

9. 建议的现场演示顺序

1. 打开首页，说明四项统计来自同一个 `Hospital` 内存模型。
2. 在医生管理新增医生，进入排班，故意添加重叠时段展示冲突校验。
3. 点击医生行的“预约”，展示信号驱动的跨页面跳转和医生预选。
4. 选择日期与时段，展示余号；提交一个预约。
5. 再用相同身份证预约相同日期时段，展示重复预约拦截。
6. 到预约查询页按姓名或预约号搜索，打开详情并演示改约或退号。

7. 返回首页，展示统计和近期预约已经刷新。

8. 最后演示导出，并说明 `QDataStream + QSaveFile` 的持久化方案。

演示前应准备：至少一位带未来排班的医生、一个容量较小的时段，以及格式正确但非真实个人信息的测试身份证号和手机号。

10. 答辩速记

- 核心类： `Hospital`；共享方式：各页面持有同一个 `Hospital&`。
- 继承关系： `Doctor/Patient → Human`；没有运行时多态。
- 排班冲突：同日且两个半开区间相交。
- 容量控制：医生 + 具体日期 + 时段的预约计数与容量比较。
- 防重复：预约号全局唯一；身份证在同一日期同一时段不能重复。
- 预约号： `AyyMMddNNN`，每天最多搜索 999 个编号。
- 删除医生：级联删除关联预约。
- 文件：医生、预约两个二进制文件；先加载医生，再加载预约。
- 保存安全：单文件通过 `QSaveFile` 原子替换，但双文件还不是整体事务。
- 刷新机制：信号槽 + 页面 `refresh()` + 共享模型。
- 测试重点：模型规则、序列化往返、导入导出；GUI 自动化尚未覆盖。
- 最大改进点：把改约封装进 `Hospital`，统一执行所有业务校验并提供事务式回滚。